第5章 第9节 文档分块实战：Java编程规范问答

- 来源: https://class.imooc.com/lesson/5273#mid=104921
- 阶段: 阶段二: 【实战入门】大模型开发与知识接入
- 周: 第5周 RAG：知识接入技术
- 模块: 第一个RAG应用：游戏知识问答系统
- 课节: 图文 1-3 文档分块实战：Java编程规范问答

1-3 文档分块实战：Java编程规范问答

第5章 第9节 文档分块实战：Java编程规范问答

> [!TIP] 阅读指南 上一节我们学习了文档分块的理论和策略，现在用一份真实的PDF文档——《阿里巴巴Java开发手册》，来体验完整的RAG构建流程：从PDF提取、文本清洗、分块策略对比，到最终入库查询。

9.1 准备工作：从PDF到可用文本

Chunking分块目标

场景：做一个能回答Java编程规范问题的智能助手

```
用户问："变量命名有什么规范？"
系统检索规范手册，给出准确答案："代码中的命名均不能以下划线或美元符号开始..."

1
2
```

数据源：阿里巴巴Java开发手册（PDF，37页，约4万字）

挑战：

- PDF格式，不是纯文本
- 有页眉页脚、页码等干扰信息
- 如何切分才能保证检索准确?

第一步：提取PDF文本

安装工具：

```
pip install pdfplumber

1
```

install

提取代码：

```
# 使用pdfplumber提取PDF文本
with pdfplumber.open(pdf_path) as pdf:
    for page in pdf.pages:
        all_text += page.extract_text()

print(f"提取完成，共 {len(all_text)} 字符")
```

> [!TIP] 完整源码参考：samples/chunking/extract_pdf.py

运行结果：

> 提取完成，共40,109 字符，总行数: 1,204

第二步：清洗文本

直接提取的文本包含很多干扰信息：

```
阿里巴巴Java开发手册
--------浏览时请使用PDF 左侧导航栏------
.......
```

这些都应该去掉。

清洗策略：

```
def clean_text(text):
    # 1. 删除页眉页脚
    text = 按行处理，跳过匹配的行

    # 2. 删除目录、表格、法律声明等固定区域
    根据行号跳过指定范围

    # 3. 压缩多余空行
    text = re.sub(r'\n{4,}', '\n\n\n', text)

    return text.strip()
```

效果：原始 40,109 字符 → 清洗后 36,495 字符，节省 9.0%

> [!TIP] 完整源码参考：samples/chapter5/chunking/clean_text.py

9.2 摘取测试文本

4万字的文本太长了，我们不需要全部，摘取一个有代表性的片段即可。

选择标准

好的测试段应该：长度适中（2000-3000字），结构清晰（有章节、有条目），内容完整（包含多个独立规范），有挑战性（有上下文依赖的内容）。

我们的选择：命名风格+常量定义

从第4-6页提取，包含：

- 16条命名规范
- 5条常量定义规范
- 正例、反例、说明齐全
- 总计约3000字

```
# 提取指定页面的文本
with pdfplumber.open(pdf_path) as pdf:
    for page_num in [3, 4, 5]:  # 第4-6页
        test_text += pdf.pages[page_num].extract_text()

test_text = clean_text(test_text)  # 清洗

print(f"测试文本长度：{len(test_text)} 字符")
```

> [!TIP] 完整源码参考：samples/chunking/extract_test_data.py

> 运行结果 测试数据: 3,389 字符

运行结果

测试数据: 3,389 字符

9.3 分块策略对比

现在我们有了3000字的测试文本，包含完整的命名规范和常量定义。其实，对于现在要测试的文本，有很明显的结构化标记，按照规范条目来拆分是最合适的。但是为了让同学们体验到不同chunking的结果，我们还是用了4种方式来切分。

> [!TIP] 完整源码参考：samples/chunking/chunking_strategies.py 可以不看源码，但是务必看一下切分结果。 切分结果参考：samples/chunking/chunking_results/

固定大小分块

思路：简单粗暴，按固定字符数切分，设置 50 字符重叠避免截断。

切分结果：共 8 个 chunk，平均大小 467 字符，大小范围 239-500 字符。

主要问题：

- 句子截断：如“TAPromotion”被切成“T”和“APromotion”

句子截断

- 规范条目被拆散：第 8 条规范的说明被截断，重叠部分造成内容冗余

规范条目被拆散

- 无视章节结构：“命名风格”和“常量定义”可能混在一个 chunk 里

无视章节结构

递归分块

思路：优先按大分隔符（章节）切，切不下再按小分隔符（段落、句子），并尝试合并小片段。

切分结果：共 6-10 个 chunk（优化后），平均大小 400-500 字符，大小范围 100-600 字符。

主要优势：

- 尊重文档结构：章节标题可以单独成块

尊重文档结构

- 保持规范完整性：每条规范不会被切断

保持规范完整性

- 自适应合并：能自动合并多个小片段到合理大小

自适应合并

结构化分块

思路：识别文档的原生结构（章节、条目），按结构切分。对于 Java 开发手册，用正则识别“1.【强制】”这种模式。

切分结果：共 20 个 chunk，平均大小 168 字符，大小范围 67-470 字符。

核心优势：

- 语义完整性最佳：每个 chunk 都是一条完整的规范条目

语义完整性最佳

- 检索精准度高：用户问“变量命名规范”，直接返回对应条目

检索精准度高

语义分块

思路：用 Embedding 检测语义边界，在语义转折处切分。计算相邻句子的余弦相似度，低于阈值则切分。

切分结果：共 30 个 chunk，平均大小 111 字符，大小范围 20-722 字符。

特点分析：

- 最智能：能自动识别话题转换，如 Chunk 2 单独是一句说明（29 字符）

最智能

- 大小不均匀：有的 chunk 只有20字符，有的却有722字符

大小不均匀

- 有成本：需要调用 Embedding API，对每个句子向量化

有成本

- 速度较慢：3000 字约 100 个句子，需要 100 次 Embedding 调用

策略选择结论

对于 Java 开发手册 这种有明确条目编号的文档，结构化分块是最佳选择：

策略	Chunk数量	平均大小	语义完整性	适用场景
固定大小	8个	467字符	★ 差	散文
递归分块	6-10个	400-500字符	★★★ 中等	层次结构文档
结构化	20个	168字符	★★★★★ 最佳	条目化文档
语义分块	30个	111字符	★★★★ 好	连贯性文档

9.4 完整流程：从PDF到问答系统

现在我们把整个流程串起来，做一个完整的 Java 编程规范问答系统。

项目结构

完整的项目结构如下（详见 samples/chapter5/chunking/README.md）：

```
chunking/
├── 数据准备
│   ├── extract_pdf.py              # PDF文本提取
│   ├── extracted_text.txt          # 原始提取结果
│   ├── clean_text.py               # 文本清洗
│   └── cleaned_text.txt            # 清洗后文本
│
├── 分块实验
│   ├── extract_test_data.py        # 提取测试数据（命名+常量章节）
│   ├── test_data.txt               # 测试数据（3,389字符）
│   ├── chunking_strategies.py      # 4种分块策略对比实现
│   └── chunking_results/           # 分块结果输出
│       ├── 1_fixed_size.txt        # 固定大小分块结果（8个chunk）
│       ├── 2_recursive.txt         # 递归分块结果（99个chunk）
│       ├── 3_structure_based.txt   # 结构化分块结果（20个chunk）
│       ├── 4_semantic.txt          # 语义分块结果（30个chunk）
│       └── 分析报告.md              # 4种策略详细对比分析
│
└── 完整问答系统
    └── java_qa_system/             # 离线/在线分离架构
        ├── extract_sample_data.py  # 提取示例数据（前4章节）
        ├── sample_data.txt         # 示例数据（8,679字符）
        ├── offline_build_kb.py     # 离线：构建知识库
        ├── online_query.py         # 在线：问答查询
        ├── kb/                     # 向量数据库（运行后生成）
        └── README.md               # 完整系统说明
```

> 说明：这是本书第一次给出完整的项目结构，以后各节不再在书中重复列举。本书包含多个小项目和一个大项目。对于复杂的大项目，同学们可以直接查看源码中的 README.md 文件，那里有更详细的结构说明、文件用途和快速开始指南。

我们在之前的章节讨论过，RAG大致可以分为两个阶段：离线阶段和在线阶段。我们首先来构建离线阶段。

离线流程：构建知识库

核心步骤：

```
# 1. 加载示例数据（为了节约API成本，我们只用前4个章节作为示例）
with open('sample_data.txt', 'r') as f:
    text = f.read()  # 8,679字符（前4个章节）

# 2. 结构化分块（通过前面的讨论，我们确定使用结构化分块）
chunks = structure_based_chunking(text)  # 50个chunk

# 3. 配置Embedding
qwen_ef = OpenAIEmbeddingFunction(
    api_key=API_KEY,
    api_base="https://dashscope.aliyuncs.com/compatible-mode/v1",
    model_name="text-embedding-v4"
)

# 4. 创建向量库
client = chromadb.PersistentClient(path="./kb")
collection = client.create_collection(
    name="java_spec",
    embedding_function=qwen_ef
)

# 5. 分批添加文档（每扁10个）
for i in range(0, len(chunks), 10):
    batch = chunks[i:i+10]
    collection.add(documents=batch, ids=[...])
```

> 说明：完整的PDF有216个chunk（约37页），但为了节约Embedding API成本，我们只用前4个章节作为示例数据（sample_data.txt），生成50个chunk。如果你需要完整的知识库，可以修改offline_build_kb.py中的DATA_FILE为cleaned_text.txt。

> [!TIP] 完整源码参考：samples/chunking/java_qa_system/offline_build_kb.py

在线查询：问答系统

```
# 1. 加载知识库
collection = load_knowledge_base()

# 2. 向量检索
results = collection.query(
    query_texts=[question],
    n_results=3  # Top-3相关文档
)

# 3. 构建上下文
context = "\n\n".join([
    f"【参考{i+1}】\n{doc}"
    for i, doc in enumerate(results['documents'][0])
])

# 4. 调用大模型生成答案
prompt = f"""你是Java编程规范专家，请基于参考资料回答问题。

参考资料：
{context}

问题：{question}
"""

response = client.chat.completions.create(
    model="qwen-plus",
    messages=[{"role": "user", "content": prompt}],
    temperature=0.3
)

return response.choices[0].message.content
```

> [!TIP] 完整源码参考：samples/chunking/java_qa_system/online_query.py

运行效果

离线构建向量数据库：

```
$ python offline_build.py
============================================================
离线处理：构建Java编程规范知识库
============================================================

[1/4] 加载示例数据...
✓ 加载完成，共 8679 字符

[2/4] 结构化分块...
✓ 切分完成，共 50 个chunk
  平均大小：173 字符/chunk

  示例chunk:
    Chunk 1: 1. 【强制】代码中的命名均不能以下划线或美元符号开始，......
[3/4] 配置Embedding函数...
✓ 使用模型: text-embedding-v4

[4/4] 构建向量库...
  清理旧数据...
  处理第 1/5 批（10 个chunk）...
  ......
  处理第 5/5 批（10 个chunk）...

知识库构建完成!
```

在线查询：

```
$ python online_query.py

============================================================
问题：java左大括号应该换行还是不换行？
============================================================

[1/2] 检索相关规范...
找到 3 条相关规范

检索来源：
  [1] 相似度：54.32%
      1. 【强制】大括号的使用约定。如果是大括号内为空，则简洁地写成{}即可.....
  [2] 相似度：37.96%
      5. 【强制】采用4个空格缩进，禁止使用tab字符......
  [3] 相似度：33.17%
      3. 【强制】if/for/while/switch/do等保留字与括号之间都必须加空格。

[2/2] 生成答案...
答案：
根据《Java编程规范》【参考资料1】中的强制规范：如果是大括号内为空，则简洁地写成{}即可，不需要换行；如果是非空代码块则：
1）左大括号前不换行。
2）左大括号后换行。
同时，【参考资料2】中的正例也体现了该规范：
if (flag == 0) {
    System.out.println(say);
}

反例：
if (flag == 0)
{
    System.out.println(say);
}
综上，Java中左大括号应紧跟在行尾不换行，其后内容换行。
```

上述输出结果分为两部分，一部分是查询向量数据库的资料，另一部分是将资料送给大模型的输出结果。

9.5 下一节预告

在这一节中，我们从 PDF 提取到文本清洗，从分块策略对比到最终的问答系统，完成了一个完整的 RAG 应用。但检索结果的准确性还有提升空间。

比如，用户问"变量命名规范"，系统可能返回了 3 条相关规范——但排在第 1 位的不一定是最准确的；有时候检索到的内容过于简略，缺少上下文；甚至有些相关但重要的信息没有被检索到。

下一节《RAG 检索优化：让答案更准确》将带你深入了解如何提升检索质量：从 Chunk 大小调优、元数据过滤，到重排序（Rerank）、混合检索，让你的 RAG 系统从"能用"升级到"好用"。

9.6 ■ 学点英语

中文	English	音标	说明
PDF 文本提取	PDF Text Extraction	/piː diː ef tekst ɪkˈstrækʃn/	从PDF文件中提取纯文本内容的技术
文本清洗	Text Cleaning	/tekst ˈkliːnɪŋ/	去除页眉页脚页码等干扰信息，保留纯内容
结构化解析	Structured Parsing	/ˈstrʌktʃərd ˈpɑːrsɪŋ/	识别并保留文档层级结构（标题、条目、标签）
正例与反例	Positive & Negative Examples	/ˈpɑːzətɪv ənd ˈnegətɪv ɪg ˈzæmplz/	技术规范中展示正确做法和错误做法的对照示例